



10. Autenticação

SCC0219 – Introdução ao Desenvolvimento Web

Prof^a. Dr^a. Bruna C. Rodrigues da Cunha

brunaru@icmc.usp.br

Autenticação

- **Autenticação de usuário**: forma de provar uma identificação de usuário garantindo que ele é quem afirma. É o processo de verificação de uma identidade.
- **Autorização**: processo de segurança que determina o nível de acesso de um usuário ou serviço. Usamos a autorização para dar aos usuários ou serviços permissão para acessar alguns dados ou realizar uma determinada ação. Depende da autenticação.



Autenticação: Níveis

Nível de segurança varia de acordo com os métodos e ferramentas implementados!



POUCO SEGURO

- Permite senhas simples
- Senhas salvas no banco sem hashing
- Envio de senha em todas as requisições
- Login sem timeout
- Troca de dados não criptografada



MUITO SEGURO

- Exige senhas complexas
- Senhas salvas com hashing e sal
- Uso de id de sessão ou token
- Troca de dados criptografada (HTTPS + SSL)
- Login timeout



1. Exigir Senhas Complexas

- Essa verificação pode ser feita na interface do usuário (*front-end*) com JS
- Usar uma expressão regular para verificar se a senha criada apresenta as características exigidas, exemplo:
 - ✓ Pelo menos 8 caracteres
 - ✓ Pelo menos um número
 - ✓ Pelo menos uma letra em maiúsculo/minúsculo
 - ✓ Pelo menos um caractere especial

```
let senha = '34a$rc0P'
```

```
let regex = /^(?=.*[A-Za-z0-9@#*$%^&+=])(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@#*$%^&+=])(?=.*{8,}).*$/g
```

```
if(senha.match(regex))  
    console.log('ok')
```

2. Salvar Senha no BD com *hashing*

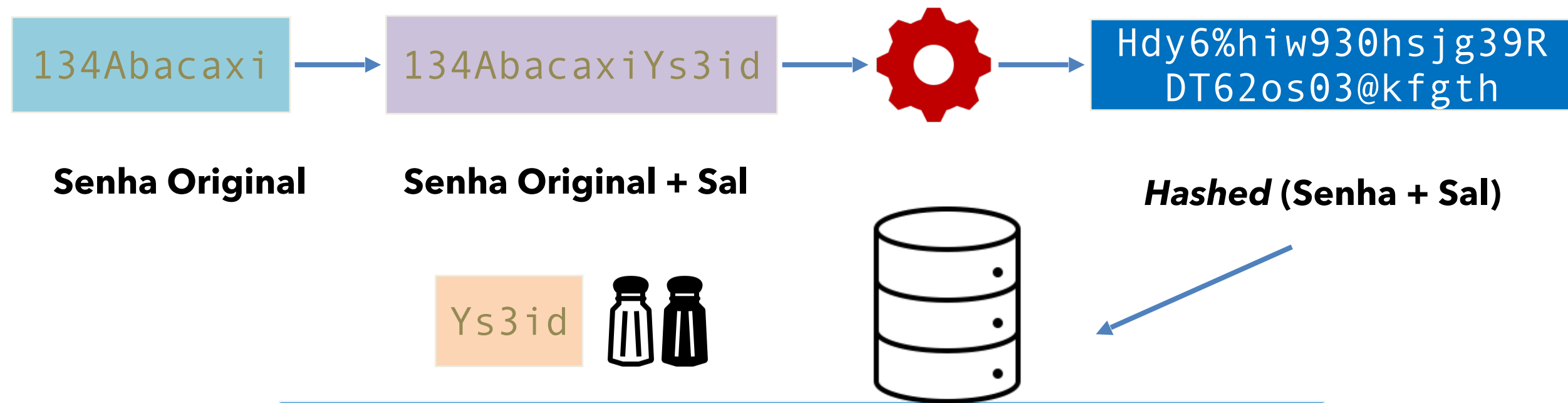
- Salvar a senha sem nenhum *hashing* no banco é uma péssima prática: qualquer pessoa com acesso (devido ou indevido) ao banco conseguirá ver a senha!
- Por questões de segurança, nem mesmo um administrador deve ser capaz de ver as senhas dos usuários.
- O *hashing* é um processo mais seguro que a criptografia.
 - O objetivo da criptografia é criptografar e descriptografar.
 - Já no *hashing*, depois de codificado, não é possível voltar ao valor original: só é possível comparar se um determinado valor corresponde ao valor criptografado.

2. Salvar Senha no BD com *hashing*

O **sal** é necessário para gerar um **valor único** de *hashing*!

O sal deve ser gerado de forma **aleatória**

Algoritmo de *hashing*



2. Salvar Senha no BD com *hashing*

```
import bcrypt from 'bcryptjs'

// hashing da senha
const salt = bcrypt.genSaltSync()
const hashedPassWord = bcrypt.hashSync(request.body.password,
salt)

// cadastra usuario
User.create({
  email: request.body.email,
  password: hashedPassWord,
})
```

2. Salvar Senha no BD com *hashing*

```
import bcrypt from 'bcryptjs'

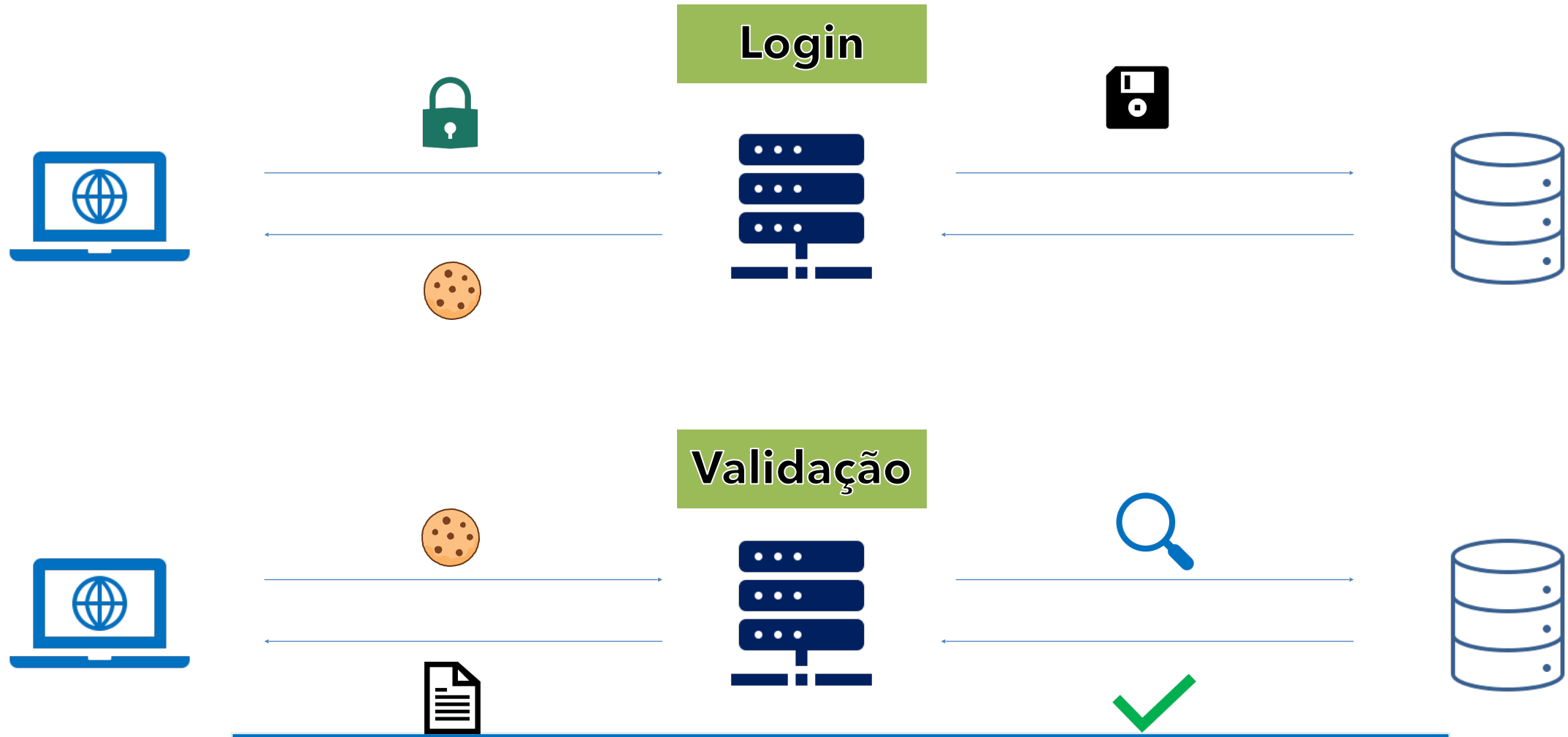
// compara senha
const isEqual = bcrypt.compareSync(request.body.senha,
user.password)

if (! isEqual) {
    response.status(401).send('Usuário e senha
inválidos!')
} else {
    // senha válida, logar no sistema
}
```

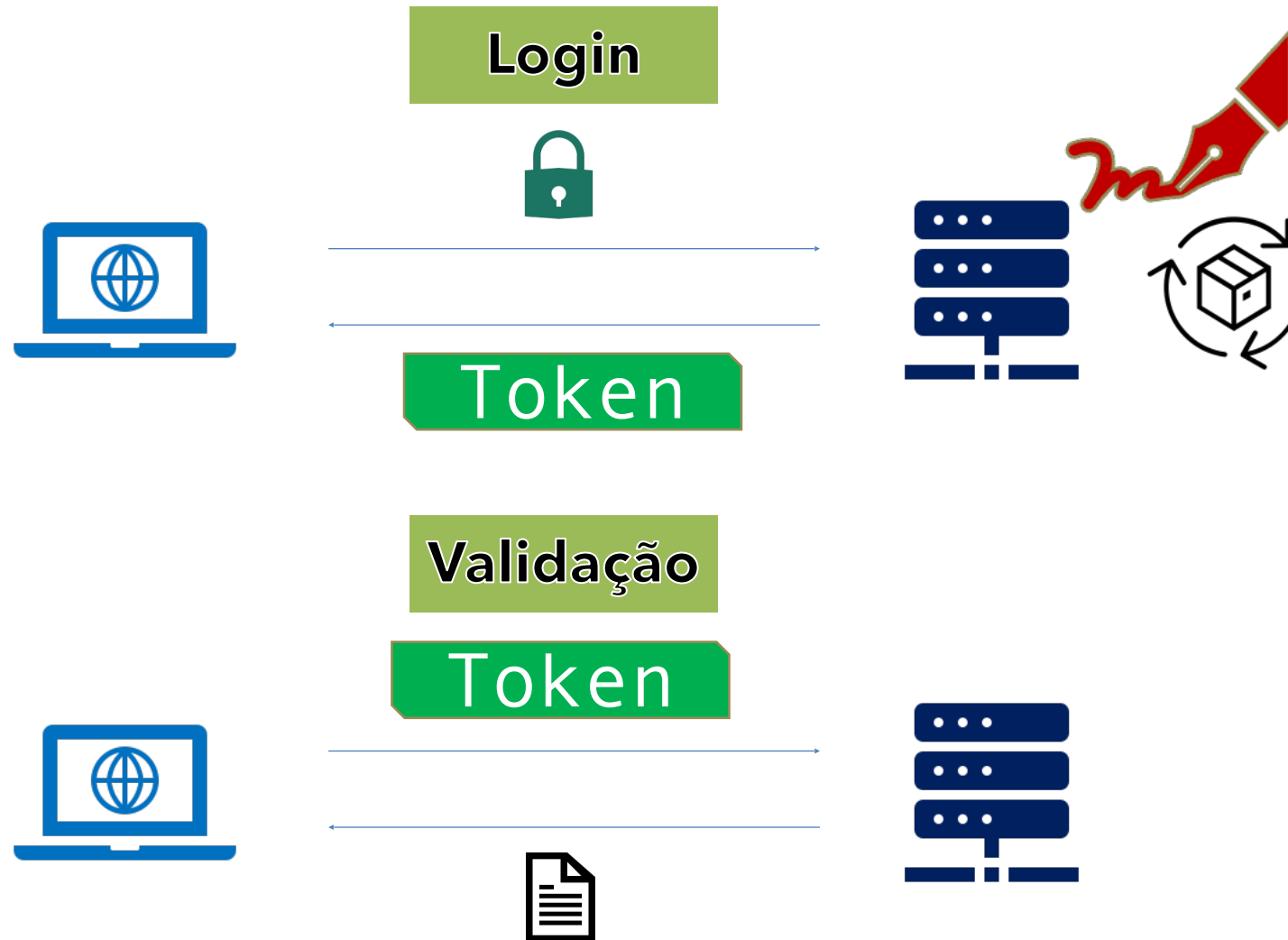
3. Baseada em Id de Sessão ou Token

- Enviar o usuário e senha em todas as requisições (autenticação básica) com certeza é uma ideia ruim.
- Por isso, o usuário deve se autenticar uma vez, enquanto o servidor mantém sua conexão ativa até que expire.
- Em uma autenticação baseada em **Id de Sessão**, o servidor salva os dados do usuário em um banco e verifica se a sessão é válida gerando um id.
- Em uma autenticação baseada em **Token**, o servidor gera um token assinado e criptografado, tendo sua verificação garantida pela assinatura. O método via Token é mais escalável pois não necessita de verificações contínuas no banco.
- Observação: em ambos os casos, o servidor deve cuidar do tempo de expiração do Id ou Token.

3. Baseada em ID de Sessão



3. Baseada em Token



3. Baseada em Token

- JSON Web Token (**JWT**) é um padrão aberto usado para compartilhar informações entre duas entidades, geralmente um cliente (*front-end/app*) e um servidor (*back-end*).
- Cada **JWT** é assinado usando criptografia para garantir que o conteúdo JSON (também conhecido como declarações JWT) não possa ser alterado pelo cliente ou por terceiros mal-intencionados.

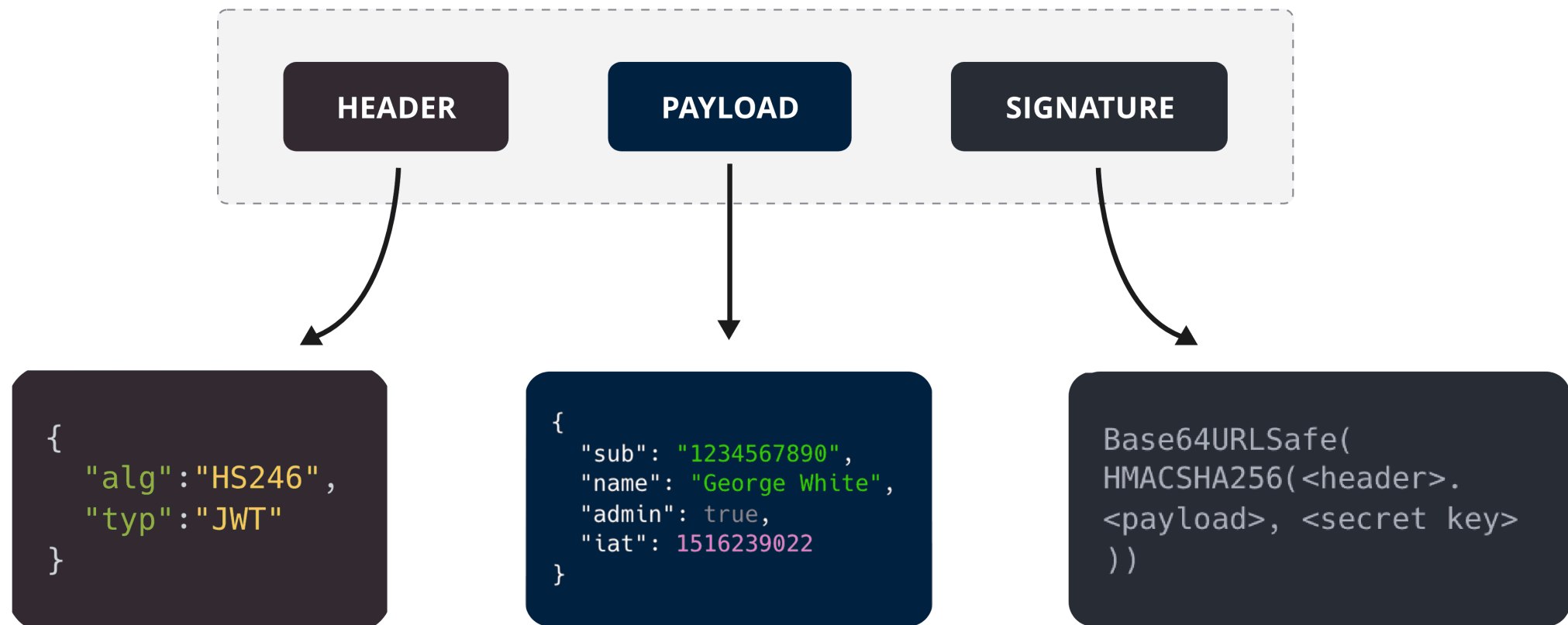
3. Baseada em Token

Um JSON Web Token (**JWT**) é estruturado em três partes: **Cabeçalho (Header)**, **Payload** e **Assinatura (Signature)**.

- a. **Cabeçalho**: consiste no algoritmo de assinatura que está sendo usado e o tipo de token, que, neste caso, é principalmente "JWT".
- b. **Payload**: contém as declarações (o objeto JSON).
- c. **Assinatura**: uma *string* gerada por um algoritmo criptográfico que pode ser usada para verificar a integridade da carga JSON.

3. Baseada em Token

Um JSON Web Token (**JWT**) é estruturado em três partes: **Cabeçalho (Header)**, **Payload** e **Assinatura (Signature)**.



3. Baseada em Token

Encoded

PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlLCJlbWFpbCI6ImJydW5hLjIwMjNAaWZzcC5lZHUuYnIiLCJpYXQiOiJlZ20DE2MzYsImV4cCI6MTY4MTc1NjQzNn0.VzK3iQ0_YKL8gXD3Gygmm74ACCLuxVjI2MT5of1HEPE
```

Decoded

EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": 15,
  "email": "bruna.2023@ifsp.edu.br",
  "iat": 1681151636,
  "exp": 1681756436
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  iS$KJkd@23i$JK79JK3at
) ☐ secret base64 encoded
```

✓ Signature Verified

SHARE JWT

3. Baseada em Token

```
import jwt from 'jsonwebtoken'

const secret = process.env['AUTH_SECRET']

const token = jwt.sign({
  sub: user.id,
  email: user.email },
  secret,
  { expiresIn: '7d' })

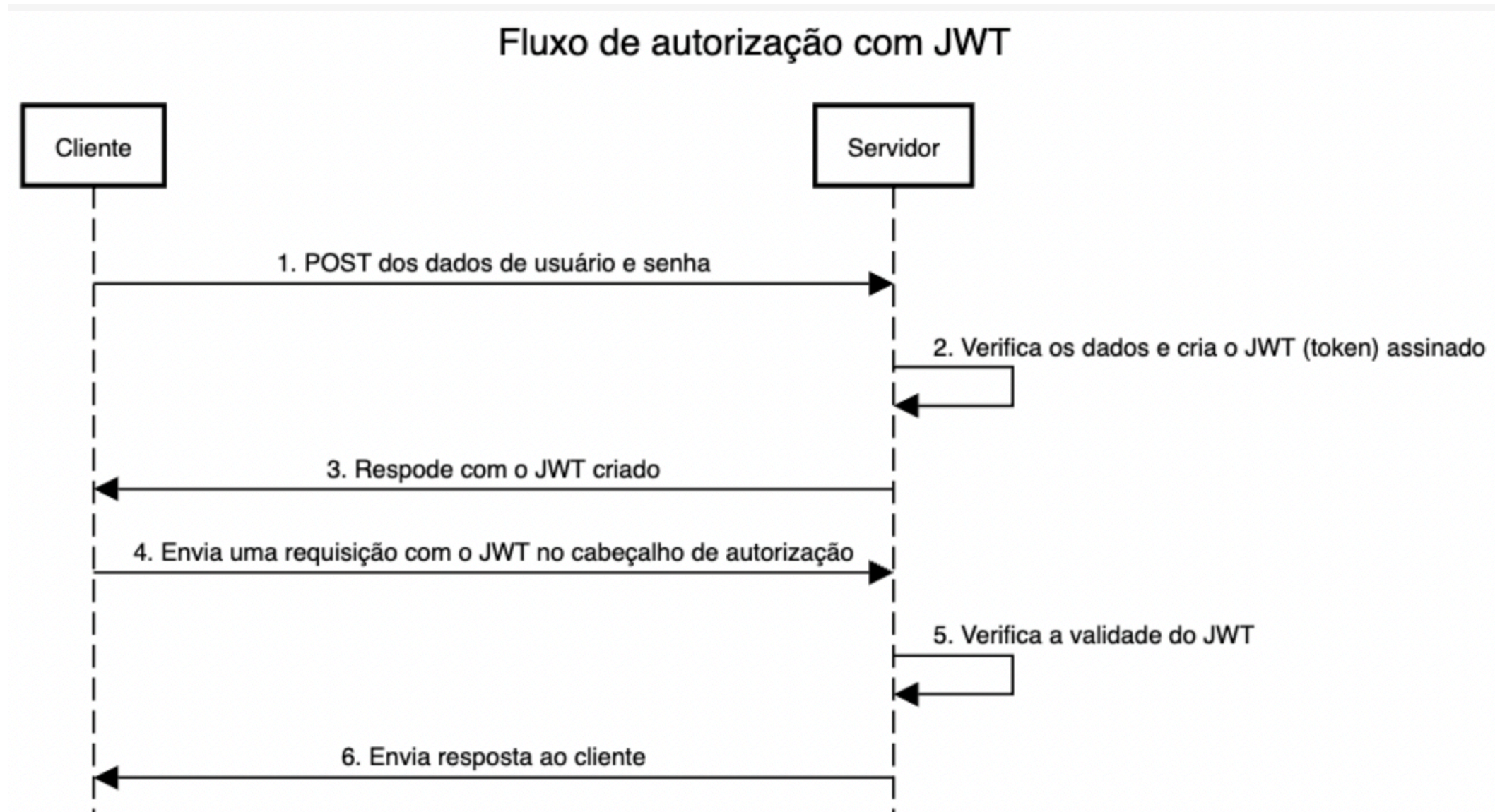
response.status(201).send({token: token})
```

3. Baseada em Token

Existe uma convenção para gerar o *payload* JWT:

- **iss** : emissor do *token*
 - **sub** : o ID de usuário
 - **email** : e-mail do usuário final
 - **email_verified** : se o usuário verificou ou não seu e-mail
 - **iat** : *timestamp* em que o JWT foi criado
 - **exp** : *timestamp* em que o JWT irá expirar
 - **admin** : se o usuário é ou não administrador
-

3. Baseada em Token

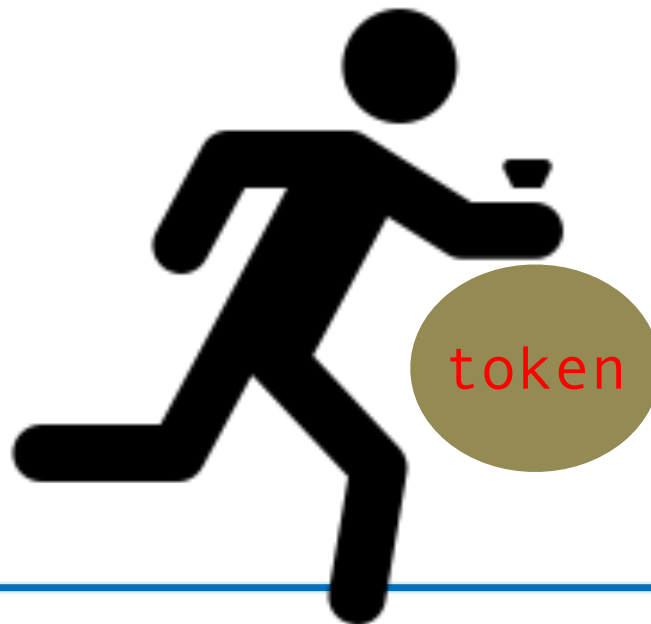


3. Baseada em Token

```
async function validarToken(request, response) {  
  const token = request.cookies.token  
  try {  
    let payload = jwt.verify(token, segredo)  
    return response.send(true)  
  } catch (e) {  
    return response.send(false)  
  }  
}
```

4. Login com *Timeout*

- É importante que seja definido um tempo de expiração de *login*, independente do fato de ser utilizado sessão ou *token*.
- Trata-se de mais uma camada de proteção (nesse caso, contra o roubo de id ou *token* de sessão).

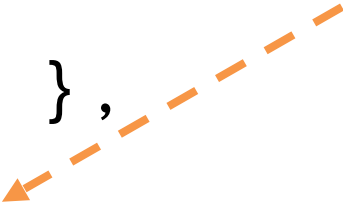


4. Login com *Timeout*

```
import jwt from 'jsonwebtoken'

const secret = process.env['AUTH_SECRET']
const token = jwt.sign({
  id: user.id,
  email: user.email },
  secret,
  { expiresIn: '7d' })

response.status(201).send({token: token})
```

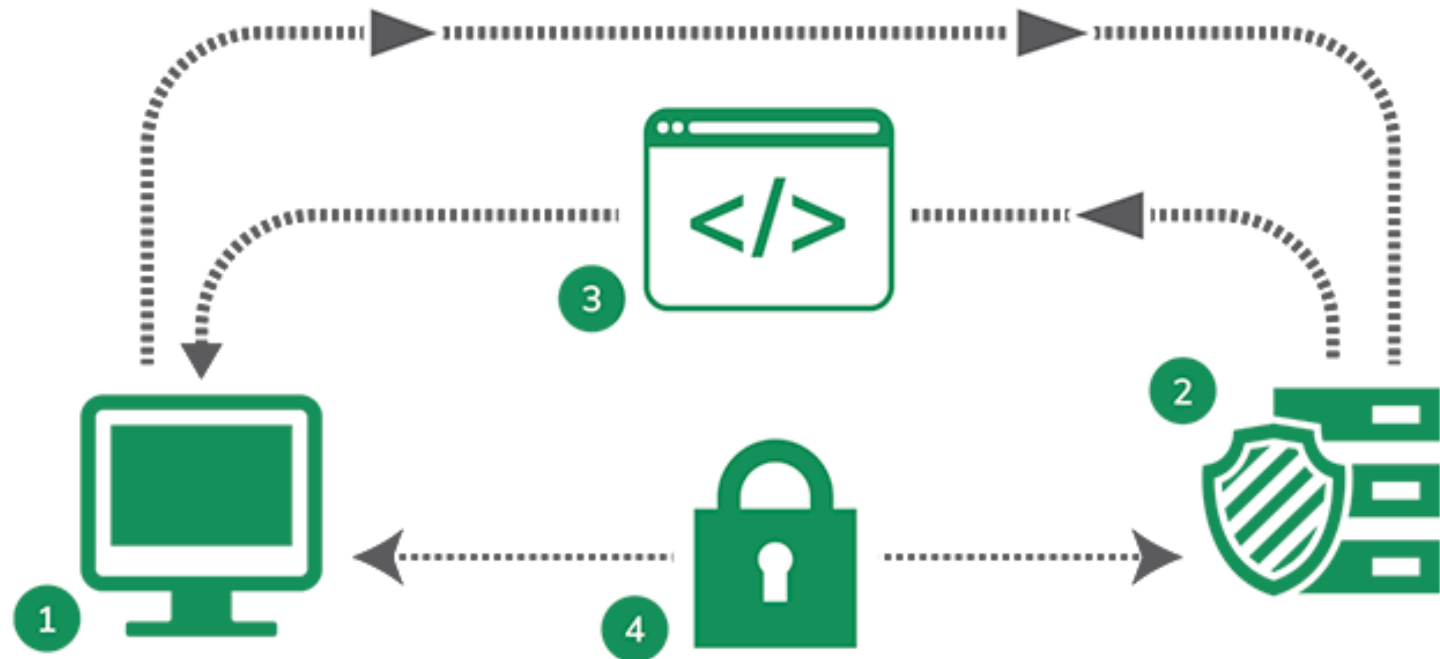


5. Troca de dados criptografada

- **HTTPS**: Protocolo de transferência de hipertexto seguro. Protocolo específico para transferir dados e mensagens Web utilizando TLS ou SSL.
- **TLS/SSL**: *Transport Layer Security* e *Secure Socket Layer* são protocolos que criam uma conexão segura entre um navegador e um servidor. O TLS é o mais novo e os principais navegadores o preferem ao SSL.
 - **Objetivos**:
 - Certificar que o servidor web que você está acessando seja operado pelo proprietário legítimo do domínio;
 - Criptografar o tráfego entre o servidor e o navegador.



5. Troca de dados criptografada



1

O visitante acessa um site com o certificado SSL instalado

2

Uma conexão SSL segura é solicitada ao servidor onde o site está hospedado

3

O servidor responde com um certificado SSL válido

4

Uma conexão segura é estabelecida entre o navegador e o servidor, permitindo a transferência de dados criptografados

5. Troca de Dados Criptografada

Symmetric encryption



Secret key



Autorização

- **Autorização** é a função de especificar direitos/privilégios de acesso a recursos.
- Ou seja, especificar se usuários possuem ou não acesso a um determinado recurso de acordo com o seu papel (*role*).
- Para adicionarmos autorização à uma aplicação Node.js com Express é simples, pois é só adicionarmos mais uma função *middleware* nas requisições.

Autenticação no Node.js

- Vamos utilizar o padrão **JWT** (autenticação baseada em *token*)
- Pacotes necessários:

```
npm i bcryptjs
```

```
npm i jsonwebtoken
```

Autenticação no Node.js

`/models/user.model.js`

```
import { Model, DataTypes } from "sequelize";
import sequelize from "../dbconfig.js";
```

```
class User extends Model {}
```

```
User.init(
{  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey:
true, },
  email: { type: DataTypes.STRING, allowNull: false, unique:
true, },
  password: { type: DataTypes.STRING, allowNull: false, },
}, { sequelize: sequelize, timestamps: false },);
```

```
export default User;
```

Autenticação no Node.js

/controllers/auth.controller.js

```

async function register(request, response) {
  if (!request.body.password || !request.body.email)
    response.status(400).send("Informe usuário e senha!");
  let user = await User.findOne({ where: {email: request.body.email} });
  if (user) response.status(400).send("Usuário já cadastrado!");

  const hashedPassword = bcrypt.hashSync(request.body.password, bcrypt.genSaltSync());

  User.create({email: request.body.email, password: hashedPassword}).then((result) => {
    const meuToken = getToken(result.dataValues.id, result.dataValues.email,);
    response.status(201).send({ token: meuToken });
  }).catch((erro) => response.status(500).send(erro));
}

```

Autenticação no Node.js

/controllers/auth.controller.js

```
import jwt from "jsonwebtoken";
import bcrypt from "bcryptjs";
import User from "../models/user.model.js";
export default { register, login, validate };

const secret = process.env["AUTH_SECRET"];

function getToken(uid, uemail) {
  const meuToken = jwt.sign({ sub: uid, email: uemail, }, secret,
    { expiresIn: "7d", },);
  return meuToken;
}
```

Autenticação no Node.js

/controllers/auth.controller.js

```

async function login(request, response) {
  if (!request.body.password || !request.body.email)
    response.status(400).send("Informe usuário e senha!");
  const user = await User.findOne({where: { email: request.body.email },});
  if (!user) response.status(400).send("Usuário não cadastrado!");

  const isEqual = bcrypt.compareSync(request.body.password, user.password);
  if (!isEqual) response.status(401).send("Usuário e senha inválidos!");

  const meuToken = getToken(user.id, user.email);
  response.status(200).json({ id: user.id, email: user.email, token:
meuToken });
}

```

Autenticação no Node.js

/controllers/auth.controller.js

```
async function validateToken(request, response, next) {  
  let token = request.headers.authorization;  
  try {  
    if (token && token.startsWith("Bearer")) {  
      token = token.substring(7, token.length);  
      const decodedToken = jwt.verify(token, secret);  
      next();  
    }  
  } catch (e) {  
    return response.status(401).send({ message:  
      "Unauthorized" });  
  }  
}
```

Autenticação no Node.js

/routes/api.routes.js

```
import authController from "../controllers/  
auth.controller.js";  
  
const router = express.Router();  
  
router.all("/clients", authController.validade);  
  
router.post("/signup", authController.register);  
router.post("/signin", authController.login);
```


Refresh Token

- Usado para diminuir o tempo de duração do token de autorização
- Passa a ter um token de autenticação de curta duração (15 minutos) e um token de atualização (refresh token) de longa duração (pode ser dias)
- O refresh token é salvo no cookies com a opção http-only
- Quando o token de autorização expira, é necessário atualizá-lo

```
npm i cookie-parser
```

Refresh Token

/controllers/auth.controller.js

```
const refreshToken = getRefreshToken(user.id, user.email)
response.cookie('refresh_token', refreshToken, {
  httpOnly: true,
  secure: true,
  sameSite: 'Strict',
  maxAge: 7 * 24 * 60 * 60 * 1000
})
```

```
const token = getAuthToken(user.id, user.email)
response.status(200).send( { token } )
```

Refresh Token

/controllers/auth.controller.js

```
async function refresh(request, response) {  
  const refreshToken = request.cookies.refresh_token  
  
  if (!refreshToken)  
    return response.status(401).send('Refresh token missing')  
  
  try {  
    const decoded = jwt.verify(refreshToken, process.env.REFRESH_SECRET)  
  
    const token = getAuthToken(decoded.sub, decoded.email)  
  
    response.status(200).send({ token })  
  } catch(e) {  
    return response.status(401).send('Invalid refresh token')  
  }  
}
```

Refresh Token

/controllers/auth.controller.js

```
function logout(request, response) {  
  response.clearCookie('refresh_token')  
  response.sendStatus(204)  
}
```

Refresh Token

/routes/api.routes.js

```
router.post('/signup', authController.register)
router.post('/signin', authController.login)
router.get('/logout', authController.logout)
router.get('/refresh', authController.refresh)
```

Refresh Token

/app.js

```
app.use(cookieParser())  
app.use(cors({  
  origin: 'http://localhost:8080',  
  credentials: true  
}))
```

Refresh Token Verificado

- A ideia é pagar o refresh token no logout
- Isso acrescenta alguns passos: salvar o refresh token em um banco e verificar se está válido
- Nesse exemplo vamos usar o banco redis, que é eficiente para o nosso exemplo. Ele precisa estar rodando na máquina.

```
npm i uuid
```

```
npm i redis
```

Refresh Token Verificado

/model/redis.js

```
import { createClient } from 'redis'

export const redis = createClient({
  url: process.env.REDIS_URL
})

redis.on('error', (err) => {
  console.error('Redis error:', err)
})

export default redis
```


Refresh Token Verificado

/server.js

```
import app from './app.js'
import { redis } from './config/redis.js'

async function bootstrap() {
  await redis.connect()

  app.listen(3000, () => {
    console.log('Servidor rodando na porta 3000.')
  })
}

bootstrap()
```

Refresh Token Verificado

/controllers/auth.controller.js

```
import redis from '../models/redis.js'
import crypto from 'crypto'

async function getRefreshToken(id, email) {
  const jti = crypto.randomUUID()
  const secret = process.env.REFRESH_SECRET
  const token = jwt.sign(
    { sub: id, email: email, jti: jti },
    secret,
    { expiresIn: '7d' },
  )
  await redis.set(`refresh:${jti}`, id, { EX: 60 * 60 * 24 * 7 })
  return token;
}
```

Refresh Token Verificado

/controllers/auth.controller.js

```

async function refresh(request, response) {
  const refreshToken = request.cookies.refresh_token

  if (!refreshToken)
    return response.status(401).send('Refresh token missing')

  try {
    const decoded = jwt.verify(refreshToken, process.env.REFRESH_SECRET)

    const exists = await redis.exists(`refresh:${decoded.jti}`)
    if (!exists)
      return response.status(401).send('Invalid refresh token')

    const token = getAuthToken(decoded.sub, decoded.email)
    response.status(200).send({ token })
  } catch(e) {
    return response.status(401).send('Invalid refresh token')
  }
}

```

Refresh Token Verificado

/controllers/auth.controller.js

```
async function logout(request, response) {  
    const refreshToken = request.cookies.refresh_token  
    if (refreshToken) {  
        try {  
            const decoded = jwt.verify(refreshToken,  
process.env.REFRESH_SECRET)  
            await redis.del(`refresh:${decoded.jti}`)  
        } catch(e) {  
            console.log(e)  
        }  
    }  
    response.clearCookie('refresh_token')  
    response.sendStatus(204)  
}
```